HELIOS Framework

User & Developer Reference Manual

Heavy Ion Linac Intelligent Orchestration

Autonomous, Safety-First Accelerator Control & Commissioning Platform

HELIOS Collaboration

Chong Shik Park (cspark7701@gmail.com)

October 15, 2026

Contents

1 System Overview and Architecture **3**
- 1.1 Introduction . . . 3
- 1.2 Core Architectural Principles . . . 3
- 1.3 Agent Taxonomy and Roles . . . 3
- 1.4 Graduated Autonomy Levels . . . 5

2 Installation and Environment Setup **6**
- 2.1 Prerequisites and Python Support . . . 6
- 2.2 Automated Environment Loader (`setup.sh`) . . . 6
- 2.3 Containerized Deployment (Docker & Docker Compose) . . . 6
  - 2.3.1 Building and Running with Docker . . . 7
  - 2.3.2 Distributed Deployment with Docker Compose . . . 7
  - 2.3.3 EPICS Host Networking and Volume Mounting . . . 7
  - 2.3.4 Executing Commands within Running Containers . . . 7
- 2.4 External Simulators and Control Tools . . . 7
- 2.5 Windows 11 Platform and Offline Deployment . . . 8
  - 2.5.1 Option 1: Windows Subsystem for Linux (WSL2 - Recommended) . . . 8
  - 2.5.2 Option 2: Native Windows 11 Environment . . . 8
  - 2.5.3 Option 3: Air-Gapped / Offline Deployment (No Internet Connection) . . . 8
  - 2.5.4 Option 4: Standalone Windows Executable Binary . . . 9

3 Operator Interfaces and CLI Usage **10**
- 3.1 Closed-Loop Orbit Correction (`helios tune-orbit`) . . . 10
- 3.2 Real-Time Web Operator Dashboard (`helios dashboard`) . . . 10
- 3.3 Interactive Operator Console (`helios chat`) . . . 11
- 3.4 Commissioning Shift Reporting (`helios report`) . . . 11
- 3.5 Tamper-Evident Audit Verification (`helios audit`) . . . 11

4 Accelerator Physics and Heavy Ion Dynamics **12**
- 4.1 Multi-Species and Multi-Charge-State (MCS) Dynamics . . . 12
- 4.2 Steerer Current Actuation and Quantization . . . 12
- 4.3 Orbit Response Matrix (ORM) Inversion . . . 12
- 4.4 RFQ 3D Vane Modulation and Cell Dynamics . . . 13
- 4.5 Solid and Liquid Metal Stripper Targets . . . 13
- 4.6 Magnetic Bunch Compressor Chicane . . . 13
- 4.7 Linac Ultra-High Vacuum and Gas Stripping Losses . . . 13
- 4.8 Space-Charge Equipartitioning and Hofmann Stability . . . 13
- 4.9 High-Power Beam Dumps and Thermo-Mechanical Shock . . . 13

5 Superconducting RF Dynamics and Conditioning **14**
5.1 Lorentz Force Detuning and Microphonics Feedforward 14
5.2 Cryoplant Heat Load Balancing 14
5.3 RF Multipactoring and Dark Current Field Emission 14
5.4 Progressive RF Conditioning Sequencer 14

6 Deterministic Safety, Fast MPS and Rollback **15**
6.1 Sub-Microsecond Fast MPS and Differential Beam Abort 15
6.2 Rollback Stack Architecture 15
6.3 Radiological Activation and RPO Clearance 15

7 Optimization, Machine Learning and Co-Simulation **16**
7.1 Reinforcement Learning Continuous Control 16
7.2 Multi-Physics Co-Simulation Bridge 16
7.3 Modular Facility Physics Twin 16

8 Testing and Continuous Integration **17**
8.1 Local Pre-Push Verification Script 17
8.2 Test Suite Execution and 100% Coverage Gate 17
8.3 Understanding Test Coverage in Accelerator Software 18

Chapter 1

System Overview and Architecture

1.1 Introduction

HELIOS (**HE**avy ion **L**inac **I**ntelligent **O**rchestration) is a distributed multi-agent artificial intelligence framework engineered for commissioning, operating, and optimizing heavy-ion linear accelerators (LEBT, RFQ, MEBT, SCL, and HEBT).

Aligned directly with the **Plan-First** paradigm established by the Lawrence Berkeley National Laboratory (LBNL) Osprey architecture, HELIOS enforces strict separation of concerns between optimization heuristics, digital twin simulation validations, deterministic safety gatekeepers, and hardware control layers.

1.2 Core Architectural Principles

1. **Pre-Flight Simulation Verification**: No setpoint is ever delivered to live hardware without prior validation through high-fidelity codes (ANL TRACK via Wine, LBNL IMPACT-Z via `lume-impact`) or sub-millisecond neural surrogates.

2. **Deterministic Safety-First Actuation**: Optimization algorithms do not talk directly to EPICS. All actuations pass through rate-of-change limiters, slew rate clamps, discrete DAC current steps, and the `SafetyAgent` rollback stack.

3. **Plan-First Execution Lifecycle**: High-level natural language goals are first compiled into structured, reproducible execution plans:

$$\begin{aligned} \text{DRAFT} \longrightarrow \text{SIMULATION_VERIFIED} \longrightarrow \text{SAFETY_APPROVED} \\ \longrightarrow \text{EXECUTING} \longrightarrow \text{COMPLETED} \end{aligned}$$

4. **Dynamic Capability Classification (Osprey Pattern)**: Instead of injecting hundreds of tools into the LLM context, HELIOS dynamically filters capabilities based on the operator's intent and domain keywords, keeping prompt sizes compact and execution deterministic.

5. **Graduated Autonomy**: Enforces clear human-in-the-loop sign-offs graduated from Advisory (Level 0) to Fully Autonomous (Level 4).

1.3 Agent Taxonomy and Roles

The framework partitions operational responsibilities among specialized autonomous agents:

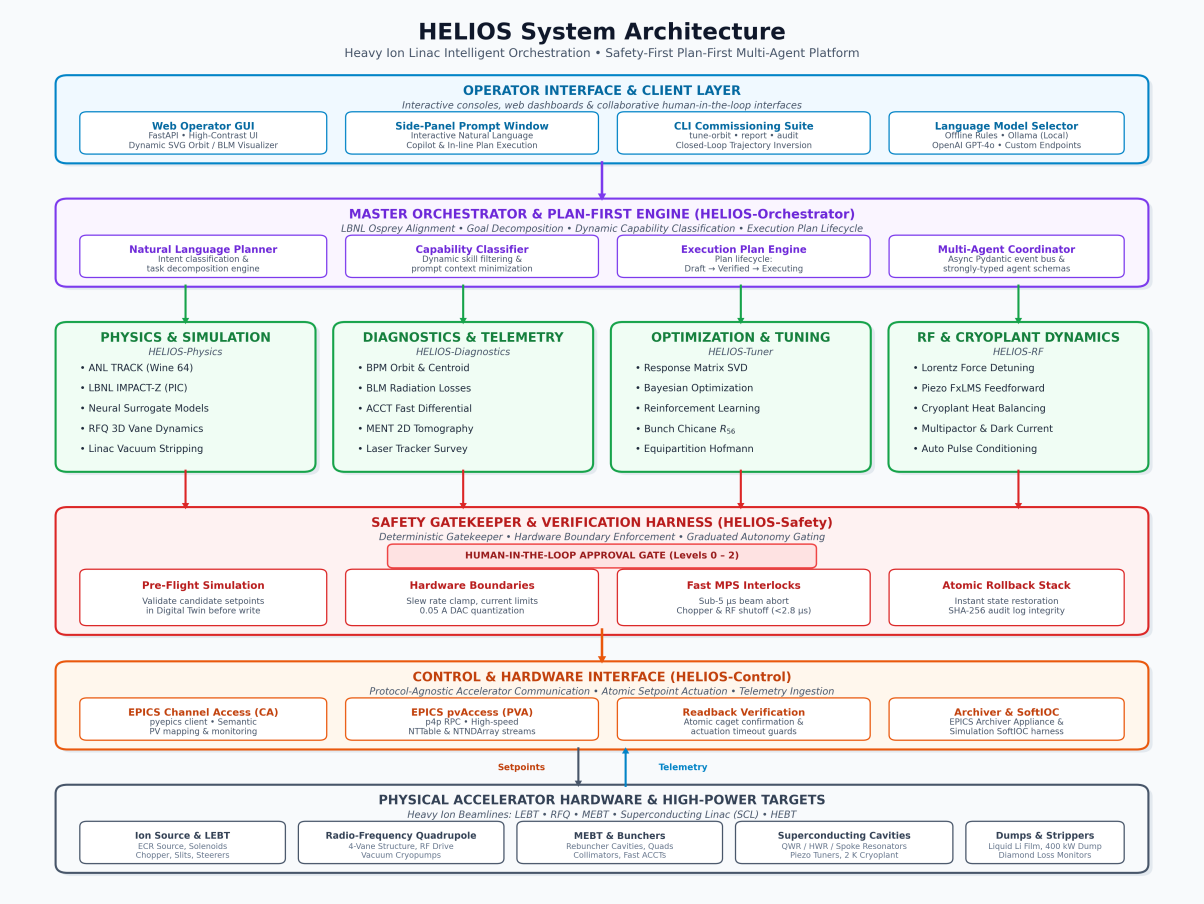


Figure 1.1: HELIOS System Architecture: Layered Plan-First multi-agent framework inspired by LBNL Osprey, showing progressive flow from operator interfaces down to physical accelerator hardware through deterministic safety gatekeepers and EPICS connectors.

1. **Master Orchestrator Agent** (`HELIOS-Orchestrator`): Decomposes high-level physicist directives into formal execution plans, manages multi-agent coordination, and interfaces with human operators.

2. **Physics and Simulation Agent** (`HELIOS-Physics`): Manages the digital twin model, executes ANL TRACK and LBNL IMPACT-Z simulations, and evaluates deep neural surrogate models.

3. **Diagnostics and Telemetry Agent** (`HELIOS-Diagnostics`): Ingests real-time signals from Beam Position Monitors (BPMs), Beam Loss Monitors (BLMs), and wire scanners; evaluates beam transmission and detects operational anomalies.

4. **Tuning and Optimization Agent** (`HELIOS-Tuner`): Executes algorithmic optimization including response-matrix SVD orbit correction, multi-objective Bayesian Optimization, and Reinforcement Learning continuous control.

5. **Safety and Verification Agent** (`HELIOS-Safety`): Serves as a deterministic gatekeeper enforcing hard software limits, slew-rate constraints, cryo-quench margins, and instant rollback stacks.

6. **Control and Hardware Interface Agent** (`HELIOS-Control`): Interfaces with the live control system via EPICS Channel Access (CA) and pvAccess (PVA) with atomic readback verification.

1.4 Graduated Autonomy Levels

HELIOS implements five graduated operational autonomy levels for human-in-the-loop gating:

Level	Name	Description	Operator Role
0	Advisory	Passive telemetry monitoring & anomaly alerts	Manual action only
1	Assisted	Pre-flight verified plans generated	Mandatory approval per setpoint
2	Supervised	Automated tuning within tight envelopes	Supervisory abort authority
3	Bounded Auto	Closed-loop optimization within boundaries	Passive monitoring
4	Fully Auto	Self-driving linac domain commissioning	Goal-setting oversight only

Table 1.1: Operational Autonomy Levels in HELIOS.

Chapter 2

Installation and Environment Setup

2.1 Prerequisites and Python Support

HELIOS is tested and verified across Python 3.10 through 3.14 (recommended: Python 3.11 in Conda/Miniforge).

```
# Clone repository
git clone https://github.com/cspark/helios.git
cd helios

# Create and activate conda environment
conda create -n helios python=3.11 -y
conda activate helios

# Install in editable mode with all dependencies (dev, rl, ml, epics,
    dashboard, ontology)
pip install -e ".[all]"

# Or select specific optional extras:
pip install -e ".[dev,ml,dashboard,ontology]"
```

2.2 Automated Environment Loader (`setup.sh`)

HELIOS provides an automated shell script `setup.sh` that automatically detects Conda/Miniforge or local virtual environments (`.venv`), exports project roots, and configures external accelerator simulation tool paths:

```
# Source the environment configuration
source setup.sh
```

This exports `PROJECT_ROOT`, `HELIOS_ROOT`, `TRACK_ROOT`, and `TRACK_EXE_PATH`, ensuring seamless execution of physics solvers across all subsystems.

2.3 Containerized Deployment (Docker & Docker Compose)

HELIOS provides a production-ready, multi-stage `Dockerfile` and `docker-compose.yml` for isolated containerized execution in control rooms and cluster environments without manual local dependency installation.

2.3.1 Building and Running with Docker

The container packages Python 3.11, Wine 64-bit (for ANL TRACK ray-tracing simulation), build toolchains, and EPICS networking:

```
# Build the production Docker image
docker build -t helios-orchestrator:latest .

# Run the container launching the Web Dashboard
docker run -d --name helios-app -p 8000:8000 helios-orchestrator:latest
```

The web operator dashboard is then accessible at http://localhost:8000.

2.3.2 Distributed Deployment with Docker Compose

HELIOS employs an asynchronous Redis message broker (`helios-redis`) on port 6379 to coordinate multi-agent operations:

```
# Launch full orchestrator and Redis stack in background
docker compose up -d

# Inspect live orchestrator logs
docker compose logs -f helios-orchestrator

# Shutdown services
docker compose down
```

2.3.3 EPICS Host Networking and Volume Mounting

When connecting to live facility hardware (e.g. RAON, FRIB) or local simulation decks, attach host networking for Channel Access broadcast discovery and mount model directories:

```
# Run with host networking and local deck mounts
docker run -it --rm --net=host \
  -v $(pwd)/examples:/app/examples \
  -v $(pwd)/docs/exec-plans:/app/docs/exec-plans \
  -e EPICS_CA_ADDR_LIST="10.0.0.255" \
  -e EPICS_CA_AUTO_ADDR_LIST="NO" \
  helios-orchestrator:latest \
  helios tune-orbit --section MEBT --autonomy 2
```

2.3.4 Executing Commands within Running Containers

Commissioning routines and interactive shells can be invoked inside an active container:

```
# Execute CLI commissioning commands
docker exec -it helios-app helios tune-orbit --section MEBT
docker exec -it helios-app helios chat
docker exec -it helios-app helios report --facility "RAON SCL1"

# Access interactive container shell
docker exec -it helios-app bash
```

2.4 External Simulators and Control Tools

- **ANL TRACK**: Heavy-ion ray-tracing and multi-charge state tracking code executed via Wine 64-bit (`wine64 track.exe`). Pre-installed in the Docker container image.

- **LBNL IMPACT-Z**: 3D space-charge Particle-in-Cell (PIC) simulation managed through `lume-impact`.
- **EPICS Base**: Channel Access and pvAccess tools (`p4p`, `pyepics`) for live communication with accelerator IOCs. Included with ports 5064 and 5065 exposed.

2.5 Windows 11 Platform and Offline Deployment

HELIOS provides flexible deployment support on Windows 11 systems:

2.5.1 Option 1: Windows Subsystem for Linux (WSL2 - Recommended)

Provides complete feature parity with Linux control systems, allowing Wine-based ray-tracing (ANL TRACK), EPICS Channel Access networking, and high-performance ML routines:

```
# Install WSL2 Ubuntu (PowerShell as Administrator)
wsl --install -d Ubuntu-22.04

# In WSL2 terminal:
git clone https://github.com/cspark/helios.git
cd helios
source setup.sh
pip install -e ".[all]"
```

2.5.2 Option 2: Native Windows 11 Environment

Core orchestration, Web Operator Dashboard, EPICS interfaces, and digital twin simulation run natively on Windows 11 with Python 3.10–3.12:

```
# In Windows PowerShell:
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -e ".[all]"
```

Note: When executing ANL TRACK natively on Windows, pass the path to the native Windows binary (`TRACKv39C.exe`) directly via `--model-dir` without Wine.

2.5.3 Option 3: Air-Gapped / Offline Deployment (No Internet Connection)

In secure accelerator control rooms disconnected from external networks:

1. **On an internet-connected machine**: Download the complete dependency wheelhouse:

```
mkdir helios_offline && cd helios_offline
git clone https://github.com/cspark/helios.git
cd helios
pip download -d ../wheels -e ".[all]"
```

2. **Transfer** the bundle to the target Windows 11 machine via authorized media.

3. **Install offline on the air-gapped machine**:

```
cd helios
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install --no-index --find-links=..\wheels -e ".[all]"
```

2.5.4 Option 4: Standalone Windows Executable Binary

HELIOS can be compiled into a standalone Windows binary (`helios.exe`) using PyInstaller, enabling operators to run CLI commands and the Web Dashboard without requiring a Python environment:

```
pip install pyinstaller
pyinstaller --name=helios --onefile --clean helios/cli/main.py
# Compiled binary located at dist\helios.exe
```

Chapter 3

Operator Interfaces and CLI Usage

HELIOS provides an ergonomic command-line interface implemented via Typer and Rich:

```
# Display CLI help and available commands
helios --help
```

3.1 Closed-Loop Orbit Correction (`helios tune-orbit`)

Executes automated trajectory correction with Orbit Response Matrix (ORM) inversion and discrete power supply current quantization:

```
# Tune MEBT section with default settings
helios tune-orbit --section MEBT

# Tune LEBT section using ANL TRACK simulation decks
helios tune-orbit --section LEBT --model-dir examples/lebt --track \
  --conversion-factor 0.168 --current-increment 0.05
```

Key parameters:

- `--section`: Accelerator beamline section (`LEBT`, `MEBT`, etc.).
- `--model-dir`, `-m`: Directory containing simulation decks (e.g., `examples/lebt` with input file `track.dat`).
- `--track / --analytic`: Toggle between real ANL TRACK simulation or analytical model.
- `--conversion-factor`, `-c`: Steerer current conversion factor (0.168 A/mrad).
- `--current-increment`, `-i`: Power supply DAC discrete step limit (0.05 A).

3.2 Real-Time Web Operator Dashboard (`helios dashboard`)

Launch the modern, high-contrast operator GUI:

```
helios dashboard --host 0.0.0.0 --port 8000
```

Navigate to `http://localhost:8000` to access:

- **High-Contrast Control-Room Theme**: Pure white background (`#ffffff`) with clear colored operational telemetry cards.

- **Dynamic Real-Time SVG Trajectory Charts**: Horizontal and vertical beam trajectory along beam coordinate z, transmission envelopes, and BLM loss heatmaps.
- **Side-Panel Copilot Prompt Window**: Natural language prompt input allowing operators to submit commands directly into the `NaturalLanguagePlanner`.
- **Language Model Selector**: Selectable backend engines (Offline Deterministic Rules, Local Ollama, OpenAI GPT-4o, or Custom Lab Endpoints).
- **One-Click Quick Actions**: Trajectory Correction, Envelope Rematching, RF Cavity Phase Scan, and Emergency Beam Abort.

3.3 Interactive Operator Console (`helios chat`)

Launch the conversational operator assistant:

```
helios chat
```

Accepts natural language operational directives (e.g., *"Correct horizontal orbit in MEBT and verify quench margins on SCL1 cavity 02"*), decomposing them into formal Osprey execution plans.

3.4 Commissioning Shift Reporting (`helios report`)

Compile electronic logbook reports:

```
helios report --facility "RAON SCL1" --output shift_report.md
```

3.5 Tamper-Evident Audit Verification (`helios audit`)

Verify SHA-256 cryptographic hash chaining across all recorded Process Variable writes:

```
helios audit --verify
```

Chapter 4

Accelerator Physics and Heavy Ion Dynamics

4.1 Multi-Species and Multi-Charge-State (MCS) Dynamics

Heavy-ion linacs frequently accelerate multi-charge state beams simultaneously (e.g., $^{238}U^{33+}$ and $^{238}U^{34+}$). The magnetic rigidity $B\rho$ is given by:

$$B\rho = \frac{p}{q} = \frac{\gamma m_0 \beta c}{q} \tag{4.1}$$

The module `helios.physics.species` models isotopic distributions and composite beam centroids across charge fractions:

$$\bar{x}(z) = \sum_q f_q \, x_q(z) \tag{4.2}$$

4.2 Steerer Current Actuation and Quantization

Dipole correctors apply a magnetic kick deflection angle θ:

$$\theta = \frac{B_y L_{\text{eff}}}{B\rho} \tag{4.3}$$

The power supply current I is calibrated linearly via $I = k_{\text{conv}} \cdot \theta$ ($k_{\text{conv}} \approx 0.168$ A/mrad). Digital-to-analog converter (DAC) discretization is modeled by:

$$I_{\text{quantized}} = \text{round}\left(\frac{I}{\Delta I}\right) \cdot \Delta I, \quad \Delta I = 0.05 \text{ A} \tag{4.4}$$

ensuring realistic simulation of power supply controller limits.

4.3 Orbit Response Matrix (ORM) Inversion

For M BPMs and N steerers, the trajectory response is $\Delta\vec{x} = \mathbf{R}\Delta\vec{\theta}$. Correction setpoints are computed via Singular Value Decomposition:

$$\mathbf{R} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T \implies \Delta\vec{\theta} = -\mathbf{V}\mathbf{\Sigma}^{+}\mathbf{U}^T\vec{x}_0 \tag{4.5}$$

4.4 RFQ 3D Vane Modulation and Cell Dynamics

The module `helios.physics.rfq` simulates cell-by-cell vane modulation ($m(z), a(z), l(z) = \beta\lambda/2$) across Radial Matching, Shaper, Gentle Buncher, and Accelerator sections with 3D field expansion:

$$U(r, \psi, z) = \frac{V_0}{2}\left[\left(\frac{r}{r_0}\right)^2 \cos(2\psi) + A_{10} I_0(kr)\cos(kz)\right] \tag{4.6}$$

4.5 Solid and Liquid Metal Stripper Targets

- **Solid Carbon Foils (`helios.physics.stripper`)**: Energy loss ΔE, multiple Coulomb scattering angular spread θ_{rms}, and radiation damage lifetime.
- **Liquid Lithium Strippers (`helios.physics.liquid_stripper`)**: Film hydrodynamics (Weber number We), multi-kW beam heating dissipation, and Langmuir evaporation rates.

4.6 Magnetic Bunch Compressor Chicane

The `helios.tuning.bunch_compressor` module models 4-dipole C-type chicanes with momentum spread $\delta = \frac{1}{\beta^2}\frac{\Delta E}{E_{\text{tot}}}$:

$$R_{56} \approx -2\theta_b^2\left(L_d + \frac{2}{3}L_b\right), \quad z_f = z_i + R_{56}\delta + T_{566}\delta^2 \tag{4.7}$$

computing optimal RF chirp $h = d\delta/dz$ for bunch compression factor $C > 1.0$.

4.7 Linac Ultra-High Vacuum and Gas Stripping Losses

The `helios.physics.vacuum` module evaluates molecular conductance, cryo-pumping speeds at 2 K, and heavy-ion electron loss cross-sections σ_{strip}:

$$I(z) = I_0 \exp\left(-\int n_{\text{gas}}(z)\sigma_{\text{strip}}\, dz\right) \tag{4.8}$$

mitigating ion-induced desorption runaway.

4.8 Space-Charge Equipartitioning and Hofmann Stability

The `helios.physics.equipartition` module calculates kinetic temperature anisotropy:

$$\frac{T_\perp}{T_\parallel} = \frac{\varepsilon_{\perp n}\sigma_\perp}{\varepsilon_{\parallel n}\sigma_\parallel} \tag{4.9}$$

and automatically rematches phase advances away from parametric resonance stopbands ($2\sigma_t - \sigma_l = 0$).

4.9 High-Power Beam Dumps and Thermo-Mechanical Shock

The `helios.physics.beam_dump` module calculates heavy-ion Bragg peak energy deposition up to 400 kW, Grüneisen hydrodynamic shock pressure $P_{\text{shock}} = \Gamma\rho_0 e_{\text{dep}}$, von Mises thermal stresses, and Tong critical heat flux cooling margins.

Chapter 5

Superconducting RF Dynamics and Conditioning

5.1 Lorentz Force Detuning and Microphonics Feedforward

Dynamic Lorentz Force Detuning under accelerating gradient:

$$\Delta f_{\mathrm{LFD}} = -K_L E_{\mathrm{acc}}^2 \tag{5.1}$$

is compensated via normalized Filtered-X Least Mean Squares (FxLMS) piezo feedforward cancellation (`helios.rf.microphonics`), yielding > 10 dB jitter reduction.

5.2 Cryoplant Heat Load Balancing

The `helios.rf.cryoplant` module evaluates BCS surface resistance $R_{\mathrm{BCS}} \propto \frac{f^2}{T} \exp(-\Delta/k_B T)$ and dynamic dissipation $P_{\mathrm{loss}} \propto V_{\mathrm{acc}}^2 / R_s$, optimizing cavity field distributions within 2.0 K cryogenic capacity.

5.3 RF Multipactoring and Dark Current Field Emission

The module `helios.rf.multipactor` models electron multipactoring resonances in coaxial couplers and cavities, alongside Fowler-Nordheim dark current field emission with bremsstrahlung radiation accumulation.

5.4 Progressive RF Conditioning Sequencer

The `AutonomousRFConditioner` automates forward power ramping with vacuum back-off interlocks ($P_{\mathrm{vac}} < 10^{-7}$ mbar) and +25% power jump breakthroughs across multipacting zones.

Chapter 6

Deterministic Safety, Fast MPS and Rollback

6.1 Sub-Microsecond Fast MPS and Differential Beam Abort

The `helios.safety.fast_mps` module monitors ACCT differential beam loss ($\Delta I > \Delta I_{\text{threshold}}$) and fast diamond BLMs, initiating LEBT chopper deflection and RFQ shutdown in < 2.8 μs.

6.2 Rollback Stack Architecture

Prior to any hardware setpoint write, the `SafetyAgent` pushes current valid states to a rollback stack. Sensor threshold trips immediately trigger atomic parameter rollbacks.

6.3 Radiological Activation and RPO Clearance

The module `helios.safety.activation` models multi-isotope Bateman decay equations across induced radioisotopes (^{56}Mn, ^{58}Co, ^{54}Mn, ^{60}Co), enforcing mandatory tunnel cooldown wait times and RPO cryptographic clearance tokens.

Chapter 7

Optimization, Machine Learning and Co-Simulation

7.1 Reinforcement Learning Continuous Control

- `LinacGymEnv`: Gymnasium environment wrapping digital twin simulations and live EPICS control.
- `LinacRLAgent`: PyTorch Actor-Critic continuous policy trained for trajectory control.
- `NeuralMPCController`: Receding-horizon model predictive controller using neural surrogates.

7.2 Multi-Physics Co-Simulation Bridge

The IPC module `helios.sim.cosim` implements `ZeroCopyCoSimBridge`, providing high-speed shared memory ring buffers for 6D bunch arrays $(N, 6)$ and 3D loss distributions synchronized with live linac repetition rates (1 to 100 Hz).

7.3 Modular Facility Physics Twin

The `helios.physics.twin` architecture provides modular digital twins, decoupled from specific accelerator facilities through runtime plugin discovery (`helios.facilities` entry points):

- **Dynamic Plugin Architecture (`registry.py`)**: Formal runtime-checkable `FacilityPlugin` protocol and `FacilityTwin` dataclass with boundary envelopes (`TwinDomain`) and zero-crash analytic fallback (`AnalyticTwinFallbackPlugin`).
- **Pre-Flight Gatekeeper (`verifier.py`)**: Sub-2ms deterministic out-of-distribution (OOD) verification checking candidate setpoints against normalized bounding boxes $[u_{\min}, u_{\max}]$ and maximum slew rates before EPICS actuation.
- **Surrogate-Accelerated Gymnasium Adapter (`env.py`)**: `LinacSurrogateGymEnv` wraps `FacilityTwin` surrogates for high-throughput RL ($\geq$ 2,000 steps/s on CPU), complete with Twiss parameter jitter and discrete DAC current quantization.
- **Differentiable Neural MPC (`mpc.py`)**: `TwinNeuralMPC` optimizes steerer trajectories via PyTorch autograd loss minimization over receding horizons.
- **Telemetry State Synchronizer (`sync.py`)**: `TwinStateSynchronizer` assimilates live BPM and wire scanner readouts into twin state vectors with MAP Twiss parameter estimation.

Chapter 8

Testing and Continuous Integration

8.1 Local Pre-Push Verification Script

Developers can test all GitHub Actions CI checks locally prior to pushing:

```
# Run local pre-push checks
./scripts/check_github_actions.sh
```

This script executes:

1. **CI Contract Verification** (`scripts/ci_contract.py`):
   - Validates GitHub Actions `ci.yml` workflow schema and execution plans.
   - Verifies zero-dependency core import boundaries.
   - Validates all CLI entrypoint command parsers.
   - Executes Ruff linting across all codebase files.
   - Enforces Mypy strict typing on safety gatekeepers, EPICS interfaces, and core abstractions.
2. **Formatting and Linting** (`ruff check .`, `black --check .`).
3. **Static Type Analysis** (`mypy helios/`).
4. **Full Test Suite with 100% Coverage Enforcement** (`pytest`).

8.2 Test Suite Execution and 100% Coverage Gate

HELIOS strictly enforces a **100.00% statement coverage gate** (`--cov-fail-under=100`, `fail_under = 100`) across all 88 Python modules in `helios/`. Run the full test suite in the active environment:

```
pytest
```

All **223 unit and integration tests** verify end-to-end functionality across all accelerator physics engines, safety gatekeepers, EPICS/PVA interfaces, digital twins, optimization algorithms, and web dashboards:

- **Total Statements**: 7,208
- **Covered Statements**: 7,208 (100.00%)
- **Missed Statements**: 0
- **Warnings**: 0 warnings across Python 3.10 through 3.14.

8.3 Understanding Test Coverage in Accelerator Software

Test coverage quantitatively measures the proportion of code executed by the automated test suite:

- **Statement Coverage**: Ensures that every line of executable code has run, eliminating unresolved symbol references, missing imports, and runtime type/attribute crashes.
- **Branch Coverage**: Ensures both normal beam operation paths and machine protection interlock trip/fallback conditions are verified.
- **Guarantees**: Eliminates unexercised dead code, detects regressions instantly on refactoring, and ensures emergency rollback paths are functional.

Detailed technical patterns and coverage inspection procedures are documented in `docs/test_coverage_guide.md`.